Estudo Comparativo de Arquiteturas de Redes Neurais Reativas e Temporais via Behavior Cloning no Jogo Hollow Knight

Heitor Leander Feitosa da Silva Mariane Elisa dos Santos Souza
{mariane.elisa,heitor.leander}@ufrpe.br
Universidade Federal Rural de Pernambuco (UFRPE)

Junho de 2026

Resumo

Este trabalho apresenta a aplicação de aprendizado por imitação via *Behavior Cloning* (BC) no treinamento de um agente autônomo capaz de enfrentar a chefe *Hornet Protector* no modo Godhome do jogo *Hollow Knight*. A base de dados utilizada compreende 200 combates demonstrados por um jogador humano, capturados em tempo de execução por meio de um *mod* que extrai o estado do combate quadro a quadro. O controle do agente é formulado como um problema de classificação multirrótulo, o que viabiliza o acionamento simultâneo de múltiplos comandos físicos. Sob o mesmo protocolo experimental, comparam-se quatro arquiteturas de redes neurais: uma rede perceptron multicamadas (MLP) reativa de quadro único, uma MLP com Janela Temporal, uma rede convolucional unidimensional (CNN 1-D) e uma rede recorrente de portas de engrenagem (*Gated Recurrent Unit* – GRU); as três últimas operam sobre uma janela deslizante de 10 quadros. A avaliação baseia-se em métricas estáticas de classificação (*precision*, *recall* e *f1-score*) e, de forma complementar, no desempenho dinâmico dos agentes na arena de jogo. Os resultados de validação indicam equivalência estatística entre as arquiteturas, registrando um *F1-score micro* estabilizado na faixa de 0,85 a 0,86, com sutil vantagem para a MLP com Janela Temporal. Empiricamente, ao longo de 60 partidas reais, todas as políticas derrotaram a chefe ao menos uma vez, com a MLP Pura e a CNN 1-D liderando em eficácia absoluta (2 vitórias cada). Notavelmente, a MLP com Janela Temporal manifestou a pior resiliência em jogo, evidenciando uma nítida dissociação em que a fidelidade de imitação estatística não atua como preditora de vitória na arena. Por fim, a variante MLP Pura destacou-se pelo melhor custo-benefício computacional, operando com aproximadamente 69 mil parâmetros, patamar dez vezes inferior à sua contraparte temporal.

Palavras-chave: aprendizado por imitação; *behavior cloning*; redes neurais; classificação multirrótulo; agentes de jogos; *Hollow Knight*.

1 Introdução

Jogos de ação 2D com chefes de padrão complexo, como *Hollow Knight*, constituem ambientes altamente desafiadores para agentes autônomos por exigirem reações rápidas,

reconhecimento de padrões de ataque e tomada contínua de decisão sobre múltiplas ações simultâneas, como movimentação, saltos, golpes e esquivas. O objetivo deste trabalho é treinar um agente capaz de derrotar a chefe *Hornet Protector* no modo de jogo Godhome. Em detrimento do aprendizado por reforço, que demanda milhões de interações e a modelagem complexa de funções de recompensa, adotou-se o aprendizado por imitação via *Behavior Cloning* (BC).

Em vez de aprendizado por reforço, que exige milhões de interações com o ambiente e o projeto cuidadoso de uma função de recompensa, adotamos o aprendizado por imitação na forma de *Behavior Cloning* (BC): o agente aprende a reproduzir as ações de um jogador humano a partir de demonstrações gravadas, tratando o problema como aprendizado supervisionado de estado para ação (1; 2). Essa escolha converte o problema de controle em um problema de classificação multirrótulo, ao qual as métricas clássicas de classificação (*precision*, *recall*, *f1-score*) se aplicam diretamente.

A principal hipótese investigada reside em verificar se o contexto temporal recente (uma janela composta pelos últimos quadros do jogo) eleva a qualidade da imitação em comparação a uma política reativa baseada estritamente no quadro atual. Para tanto, realiza-se uma comparação controlada entre quatro arquiteturas neurais (MLP Pura, MLP com Janela Temporal, CNN 1-D e GRU) sob o mesmo conjunto de dados e protocolo de treinamento.

As contribuições centrais deste trabalho dividem-se em três pilares integrados:

1. a consolidação de uma base de dados própria de demonstrações humanas, coletada por meio de um *pipeline* acoplado diretamente à memória do jogo em tempo de execução;

2. uma representação de estado compacta e normalizada do combate (posições e velocidades relativas, vida, alma e estado do chefe);

3. uma comparação controlada entre arquiteturas reativas e temporais para BC neste cenário, tanto em métricas de classificação quanto em desempenho *in-game*, com as otimizações de treino e de inferência necessárias.

2 Metodologia

2.1 Visão geral do sistema

O ecossistema é composto por dois processos concorrentes que operam de forma síncrona, comunicando-se via *named pipe* nativo do Windows. No lado do jogo, utiliza-se o *framework* BepInEx integrado a um *mod* injetável em C# desenvolvido originalmente por Christofoletti (3) e customizado para este experimento. Esse componente realiza a leitura das variáveis de memória diretamente a cada ciclo de física (`FixedUpdate`), coletando os dados de telemetria do jogador e da chefe (coordenadas cartesianas, velocidades relativas, status de vida, Alma e *flags* de animação), que são serializados e transmitidos como uma *struct* binária compacta.

No lado da Inteligência Artificial, um *script* em Python baseado em PyTorch consome o fluxo de bytes do *pipe*, converte o estado bruto em um vetor de características normalizado e submete as variáveis ao modelo neural. Na fase de atuação, os comandos preditos pela política da rede são despachados para um *gamepad* virtual via `vgamepad`, emulando fisicamente a entrada de um controle convencional. Todo o ciclo de execução, mapeamento

e gravação de dados é disparado e interrompido dinamicamente por meio da identificação do *hash* de cena correspondente à arena da Hornet (`423158243`).

2.2 Construção do Dataset e Engenharia de Atributos

A base de dados foi construída integralmente pelo grupo por meio de demonstrações de um jogador humano enfrentando a chefe Hornet. Um módulo automatizado capturou sequências de quadros contendo os estados internos do jogo e os correspondentes *snapshots* do comando físico. O conjunto final consolidou um escopo de 200 arquivos CSV de demonstrações bem-sucedidas (lutas vencidas) sob o padrão descritivo `demo_<timestamp>_hash<scene>.csv`.

Em etapa de pós-processamento, as entradas do controle físico foram binarizadas para compor o vetor alvo de treinamento. Os eixos analógicos contínuos do direcional esquerdo foram discretizados em quatro componentes digitais binários independentes por meio da aplicação de um limiar de corte de $\pm 0{,}5$. O gatilho direito e as chaves digitais dos botões A, B e X foram analogamente binarizados. Essa filtragem consolidou um espaço amostral de 8 ações binárias assíncronas (Tabela 1), que atuam como os rótulos de saída para os modelos.

Tabela 1: Espaço de ações binarizadas do agente e mapeamento de telemetria.

Índice	Comando no Jogo	Mecânica Equivalente	Variável na Telemetria
1	Gatilho Direito	*Dash* / Esquiva	`gp_trigger_right`
2	Botão A	Pulo	`gp_btn_a`
3	Botão B	Foco / Conjuração	`gp_btn_b`
4	Botão X	Ataque Básico	`gp_btn_x`
5	Direcional Esq.	Movimento à Esquerda	`gp_axis_left_x` $< -0{,}5$
6	Direcional Dir.	Movimento à Direita	`gp_axis_left_x` $> 0{,}5$
7	Direcional Cima	Direcionamento Vertical	`gp_axis_left_y` $> 0{,}5$
8	Direcional Baixo	Direcionamento Vertical	`gp_axis_left_y` $< -0{,}5$

Para o espaço de entrada, a telemetria bruta foi convertida em um vetor estruturado de 104 atributos por quadro, dimensão alcançada devido à expansão via codificação *one-hot* das variáveis categóricas de ação da chefe (`bossState`). O vetor contempla posições relativas cartesianas (Δx, Δy) normalizadas pela amplitude da arena para o intervalo $[-1, 1]$, velocidades relativas (Δv_x, Δv_y), além de métricas de vitalidade e recursos (*HP*, *Soul* e *Boss HP*) escalonadas por seus limiares máximos absolutos. Variáveis booleanas de contexto físico (como colisão com o chão ou estados de invulnerabilidade) foram convertidas em sinalizações digitais 0 ou 1. Por fim, coordenadas globais absolutas foram descartadas para garantir a invariância posicional do agente na arena, forçando os modelos a convergirem suas políticas unicamente nas interações dinâmicas entre o jogador e a chefe.

2.3 Formulação do problema (*Behavior Cloning*)

O BC é formulado como classificação multirrótulo supervisionada: dado o estado x (ou a janela de estados), a rede produz um logit z_j por ação, e a probabilidade de cada ação estar ativa é obtida pela sigmoide $\sigma(\cdot)$. A função de perda é a entropia cruzada binária com logits (`BCEWithLogitsLoss`), aplicada de forma independente a cada uma das 8 saídas:

$$\mathcal{L} \;=\; -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{8}\Big[\, y_{ij}\,\log\sigma(z_{ij}) \;+\; (1-y_{ij})\,\log\big(1-\sigma(z_{ij})\big)\Big]. \tag{1}$$

Na avaliação (Seção 3.2), aplica-se a sigmoide seguida de limiar 0,5 para decidir cada botão. No agente em jogo, o limiar foi reduzido para 0,2, conforme discutido na Seção 2.5.

2.4 Arquiteturas avaliadas e motivação

Foram comparadas quatro arquiteturas sob o paradigma de Aprendizado por Imitação. Devido à natureza multirrótulo (*multi-label*) da tarefa, todas partilham uma camada de saída com 8 neurónios e ativação sigmoide , otimizada via perda *BCEWithLogitsLoss*. Enquanto a MLP Pura opera sobre um único quadro instantâneo (1×104), os demais modelos utilizam uma janela deslizante contendo o histórico dos últimos 10 quadros (10×104).

A MLP Pura ($104 \to 256 \to 128 \to 64 \to 8$, *dropout* de 0,25) atua como linha de base reativa para testar se o estado atual é suficientemente Markoviano. Como contraparte temporal ingénua, a MLP com Janela Temporal realiza o achatamento linear do histórico num vetor plano de dimensão 1040, processando-o por camadas densas ($1040 \to 512 \to 256 \to 128 \to 8$, *dropout* de 0,4). A CNN 1-D rearranja o tensor por inversão de eixos para tratar os atributos como canais, aplicando duas convoluções sequenciais ($104 \to 32 \to 64$, *kernel* 3) ao longo do tempo para capturar sinais locais de antecipação do chefe antes de mapear a decisão em camadas densas ($640 \to 128 \to 8$, *dropout* de 0,3). Por fim, a GRU introduz uma estrutura recorrente de 2 camadas com estado oculto de dimensão 128 (*dropout* de 0,3) e *gradient clipping* de 1,0; sua classificação isola o último quadro da janela para avaliar se a dinâmica de combate exige a retenção de dependências sequenciais de longo prazo.

A Tabela 2 compara o tamanho das arquiteturas.

Tabela 2: Comparativo de tamanho das arquiteturas.

Modelo	Entrada	Nº aprox. de parâmetros
MLP	1×104	69 mil
CNN 1-D	10×104	100 mil
GRU	10×104	190 mil
MLP com Janela Temporal	10×104	699 mil

2.5 Protocolo de treinamento e avaliação

O conjunto de dados composto por 200 lutas foi dividido na proporção de 80% para treinamento (160 lutas) e 20% para teste e validação (40 lutas). A amostragem foi estratificada por arquivos de combate individuais, e não por linhas isoladas da telemetria, critério essencial para evitar o vazamento de dados (*data leakage*) entre os conjuntos. O treinamento foi executado em ambiente Kaggle com aceleração por GPU utilizando a biblioteca PyTorch (4).

O processo estendeu-se por 100 épocas com lotes (*batch sizes*) de 64 amostras para treino e 512 para teste. Utilizou-se o otimizador Adam com taxa de aprendizado inicial de

0,001, *weight decay* entre 10^{-5} e 10^{-4} e o agendador `ReduceLROnPlateau` para decaimento adaptativo. Os melhores pesos foram preservados via *checkpoints* baseados na menor perda de teste, aplicando-se também o ceifamento de gradiente (*gradient clipping*) em 1,0 no modelo GRU. Por fim, o desempenho preditivo multirrótulo dos agentes foi mensurado por meio do módulo `classification_report` da biblioteca *scikit-learn* (5), extraindo as métricas de precisão, revocação e *f1-score* de forma individual e agregada.

Por fim, a análise preliminar sob o limiar estatístico padrão de 0,5 revelou um perfil de comportamento excessivamente conservador, no qual os modelos manifestavam alta precisão em detrimento de uma revocação (*recall*) reduzida, resultando na omissão de comandos essenciais de ataque ou evasão que um piloto humano executaria na mesma janela de frames. Para mitigar essa passividade, o limiar de ativação dos logits na inferência foi reduzido para 0,2 no ambiente de teste real, expandindo a taxa de acionamento das mecânicas reativas sem comprometer a estabilidade de controle dos modelos.

3 Resultados

A avaliação é dividida em duas partes: métricas de classificação no conjunto de teste (Seção 3.2) e resultados empíricos do agente jogando contra o chefe (Seção 3.3).

3.1 Treinamento e Convergência

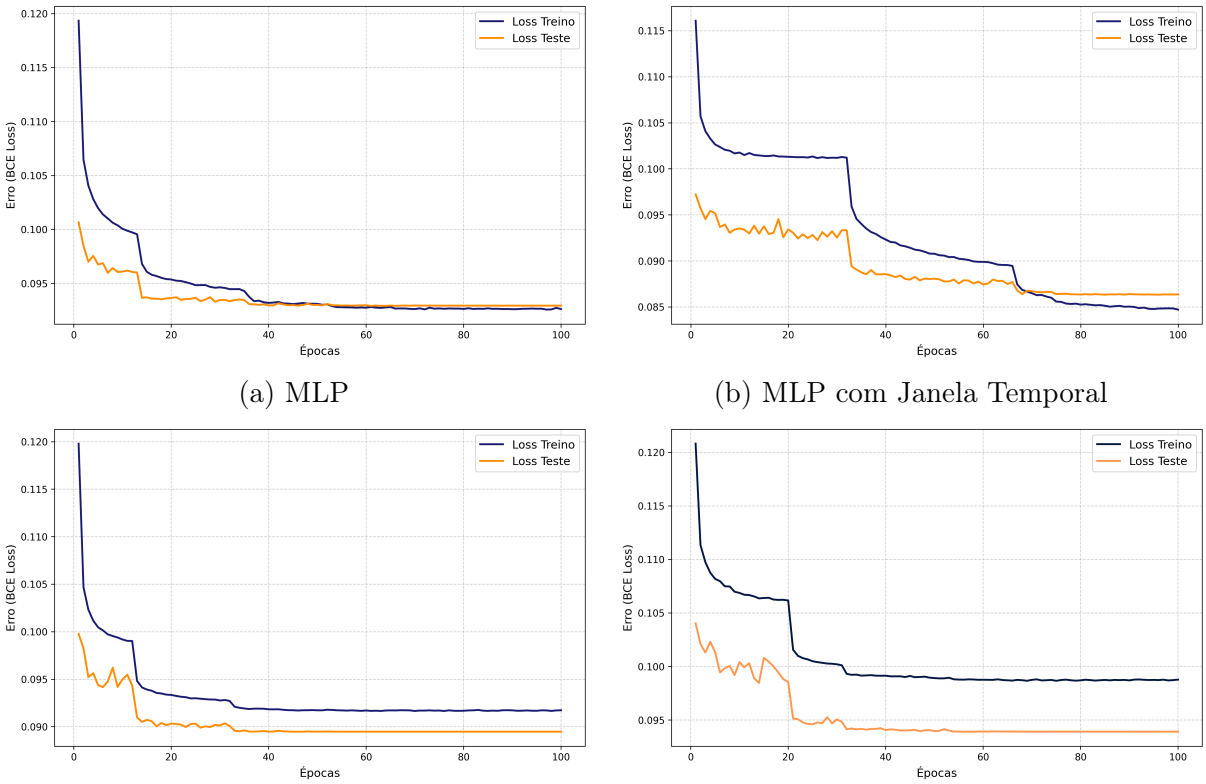
No que tange à convergência dos algoritmos, a perda de teste/validação estabiliza relativamente cedo, e o melhor modelo é preservado por *checkpoint* como descrito na Seção 2.5. A Figura 1 ilustra as curvas de aprendizado obtidas durante o treinamento final das quatro arquiteturas. Em todos os cenários, a curva de teste acompanhou a trajetória da curva de treino até a convergência assintótica, sem manifestar grande divergência ou distanciamento acentuado. Esse comportamento indica o controle efetivo do fenômeno de sobreajuste (*overfitting*), validando a eficiência das técnicas de regularização e do escalonamento da taxa de aprendizado adotados na pesquisa.

3.2 Avaliação Estática de Classificação

No cálculo de métricas estáticas básicas, os resultados mostram desempenho próximo entre as arquiteturas, mas não um empate: as quatro redes situam-se em uma faixa estreita de *f1-score micro* entre 0,85 e 0,86.A variante MLP com Janela Temporal liderou discretamente os indicadores agregados, atingindo os maiores valores de *f1-score micro* (0,86) e *macro* (0,78). Sob uma perspectiva estritamente *offline*, o contexto temporal da janela de 10 quadros trouxe, portanto, um ganho pequeno, porém consistente, sobre a política reativa de quadro único. Esse ganho modesto é coerente com a representação de estado adotada, que já inclui velocidades relativas e, assim, codifica boa parte da informação dinâmica que uma janela traria.

A Tabela 3 resume o desempenho agregado das quatro arquiteturas no conjunto de teste (40 lutas, com cerca de 128 mil ativações de ações), e a Tabela 4 detalha o f1-score por ação.

Outro padrão consistente é o perfil conservador das redes: em todas, a *precision* supera o *recall* (*precision micro* entre 0,90 e 0,91, contra *recall micro* entre 0,80 e 0,82). O botão B, por exemplo, tem *precision* entre 0,93 e 0,96 e *recall* entre 0,76 e 0,81; o ataque X tem *precision* entre 0,88 e 0,91 e *recall* entre 0,68 e 0,73.



(a) MLP

(b) MLP com Janela Temporal

(c) CNN 1-D

(d) GRU

Figura 1: Curvas de aprendizado (perda BCE de treino e de teste por época) do treino final. Em todas as arquiteturas, as perdas de treino e teste convergem juntas e se estabilizam, sem o distanciamento característico de sobreajuste; os degraus visíveis indicam às reduções de taxa de aprendizado do `ReduceLROnPlateau`.

Tabela 3: Métricas agregadas no conjunto de teste.

Modelo	Precision *micro*	Recall *micro*	f1 *micro*	f1 *macro*	f1 *weighted*	f1 *samples*
MLP	0,90	0,80	0,85	0,76	0,84	0,56
CNN 1-D	**0,91**	0,80	0,85	0,77	0,84	0,56
MLP com Janela Temporal	0,90	**0,82**	**0,86**	**0,78**	**0,85**	**0,57**
GRU	**0,91**	0,80	0,85	0,76	0,84	0,56

Tabela 4: f1-score por ação (suporte = nº de ocorrências positivas no teste).

Ação	Suporte	MLP	CNN	MLP com Janela Temporal	GRU
Gatilho direito (*dash*)	1 305	0,78	0,84	**0,85**	0,80
A (pulo)	9 149	0,90	0,90	0,90	0,89
B (*cast*/cura)	11 188	0,84	**0,86**	**0,86**	0,85
X (ataque)	11 503	0,78	**0,80**	**0,80**	0,79
Esquerda	38 107	0,90	0,90	0,90	0,90
Direita	40 942	0,91	0,91	0,91	0,90
Cima	5 315	0,30	0,26	**0,35**	0,26
Baixo	10 652	0,66	0,67	**0,68**	0,67

Esse diagnóstico evidencia que as redes operam por omissão em cenários ambíguos, preferindo abdicar do acionamento de um comando a disparar um acionamento incorreto. Essa constatação motivou a redução estratégica do limiar de inferência para 0,2 no agente em tempo de jogo, conforme detalhado na Seção 2.5, visando expandir a agressividade e a reatividade do modelo na arena.

No detalhamento por ação, a movimentação horizontal (cerca de 79 mil ocorrências) e o pulo são bem aprendidos, com *f1-score* em torno de 0,90, refletindo sua alta frequência e regularidade nas demonstrações. O ponto fraco comum em termos de métrica estática foi o direcional "Cima", com *f1-score* crítico entre 0,26 e 0,35. Esse resultado decorre diretamente do viés de demonstração do especialista humano, visto que o comando de olhar ou atacar para cima quase não foi executado durante as sessões de gravação, representando cerca de 4% do suporte total do dataset.

Duas observações se aplicam à leitura desses números. A primeira é que as métricas medem a fidelidade da imitação ao jogador humano, e não a capacidade de derrotar o chefe, avaliada na Seção 3.3: um *f1-score* alto é condição necessária, mas não suficiente, para vencer o combate, como os resultados *in-game* deixam claro. A segunda é que as janelas deslizantes do teste foram geradas sobre os dados concatenados (com perda de apenas 9 quadros no total), de modo que janelas próximas às fronteiras podem misturar quadros de lutas distintas; o efeito é marginal, mas em versões futuras a janela deve ser reiniciada a cada luta.

3.3 Resultados Empíricos e Avaliação em Jogo (*In-Game*)

Para medir a capacidade real de combate, cada modelo foi submetido a 15 partidas contra a *Hornet Protector*, totalizando um escopo experimental de 60 combates. O agente controlou o ambiente via `vgamepad` sob um limiar de decisão de 0,2. Os testes em tempo de execução foram conduzidos localmente em um processador AMD Ryzen 5 5500 com 16 GB de memória RAM. Devido à ausência de suporte à arquitetura CUDA na GPU local, o pipeline de inferência operou estritamente via CPU, dividindo os ciclos de processamento do ecossistema de controle com a execução nativa do motor do jogo.

Para a interpretação qualificada dos dados, definem-se dois critérios de desempenho dinâmico: uma "cura bem-sucedida" contabiliza cenários em que o agente completou a ação sem interrupções, indicando a escolha da janela correta de oportunidade; e um "comportamento complexo" caracteriza a sequência de comandos rápidos com resultado benéfico, como magias direcionadas e técnicas de *pogo* (atacar verticalmente para baixo saltando sobre o oponente).

A Tabela 5 reporta a contagem de vitórias e as médias por partida das variáveis monitoradas pelo ecossistema de telemetria.

Tabela 5: Desempenho *in-game*: Médias por partida.

Modelo	Vitórias	Sobrev. (s)	Dano %	Acur. ataque	Golpes sofr.	Esquivas	Curas
MLP Pura	**2/15**	67,55	55,70 %	68,49 %	11,3	1,1	3,60
MLP com Janela Temporal	1/15	55,36	51,98 %	71,91 %	10,6	0,9	2,67
CNN 1-D	**2/15**	**78,71**	**71,27 %**	**80,72 %**	12,2	**2,3**	4,53
GRU	1/15	76,83	69,71 %	71,02 %	12,5	1,8	**5,07**

A análise cruzada dos logs comprova que a longevidade dos agentes na arena é uma função direta da sinergia entre seus componentes de suporte. Constatou-se uma forte

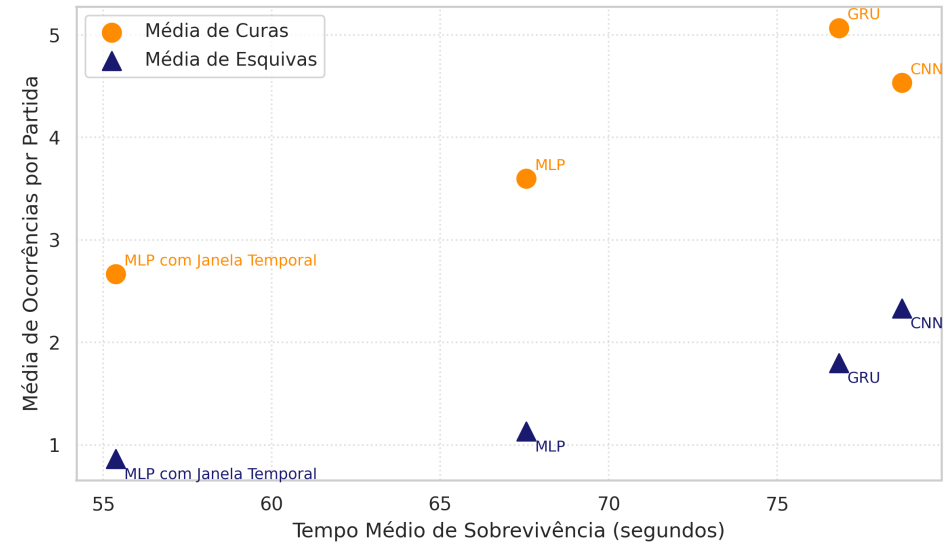


Figura 2: Relação entre o tempo de sobrevivência médio da rede e a média de curas e esquivas bem-sucedidas

correlação linear positiva entre o gerenciamento de vitalidade e a atividade evasiva em relação ao tempo total de batalha ($r = 0{,}96$ para curas e $r = 0{,}92$ para esquivas), como indicado na Figura 2.

A CNN 1-D e a GRU, que registraram os maiores índices de desvios defensivos, considerando curas e esquivas, sustentaram os tempos de sobrevivência mais elevados do experimento (78,71 s e 76,83 s). Em contrapartida, a MLP com Janela Temporal manifestou a menor atividade evasiva e menor frequência de cura, reduzindo sua resiliência defensiva.

No domínio ofensivo, os dados revelam a superioridade das arquiteturas que extraem padrões temporais profundos para acumular dano absoluto, embora se manifeste um paradoxo prático nas taxas de finalização. A CNN 1-D e a GRU demonstraram maior consistência de combate, acumulando os maiores volumes de dano percentual médio infligido à chefe ($\approx 70\%$). Contudo, a MLP Pura e a CNN 1-D alcançaram a maior taxa de sucesso absoluto (2/15 cada), em comparação com as demais (1/15 cada). Os dados confirmam que todas as arquiteturas consolidaram políticas funcionais, derrotando a chefe ao menos uma vez, enquanto a MLP com Janela Temporal registrou o pior desempenho geral do ecossistema em longevidade (55,36 s) e dano acumulado (51,98%).

No campo das mecânicas especiais, manifestou-se uma clara inversão de competências entre as redes reativas e recorrentes. A MLP Pura demonstrou a maior eficácia no disparo de feitiços, registrando 60,00% de acurácia, mostrando-se a única rede capaz de coordenar e executar com sucesso a conjuração de uma magia direcionada para baixo,, contra apenas 23,33% da arquitetura GRU. A investigação qualitativa revelou um padrão comum de dessincronização: o comando de ativação da magia ocorre na janela correta, porém manifesta um erro de orientação espacial, resultando em disparos direcionados para o lado oposto ao da chefe.

A CNN 1-D uniu o maior dano médio à maior precisão de acertos físicos básicos (80,7%) mesmo que sua precisão na conjuração de magias se aproxime do aleatório (54,7%) como apresentado na Figura 3.

Por fim, a GRU sobressaiu-se na retenção de dependências temporais estruturadas

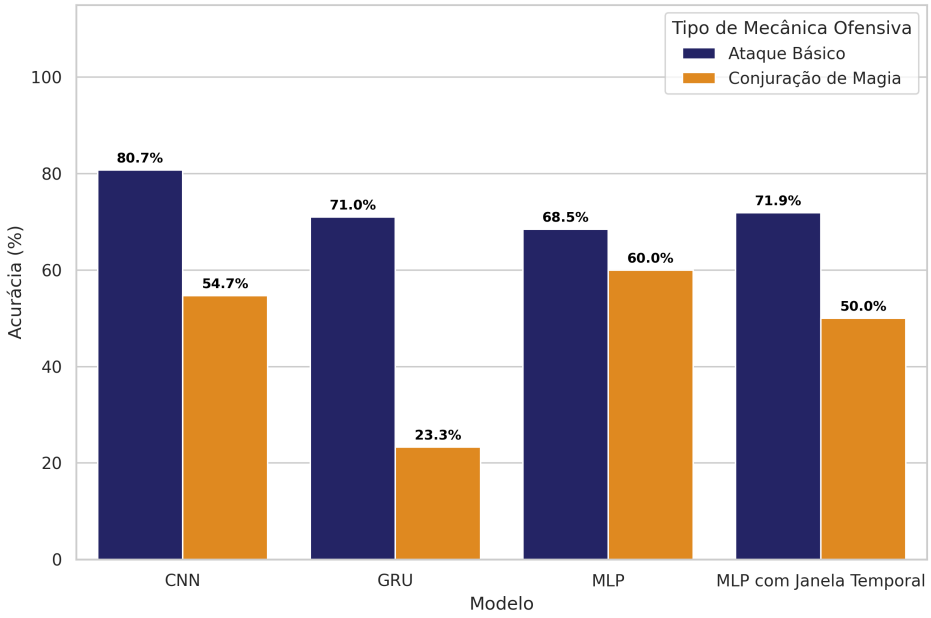


Figura 3: Comparativo de Acurácia Média dos Ataque para cada uma das arquiteturas

de longo prazo, liderando de forma isolada em comportamento sequencial complexo ao manifestar 6 execuções de comportamentos complexos com consistência estatística, seguida da CNN 1-D (3 comportamentos complexos) e, depois, das MLPs (2 comportamentos complexos cada). Contudo, o custo computacional do fluxo recorrente executado em CPU gerou um gargalo de atraso na emissão física dos comandos, fazendo com que o modelo reagisse alguns quadros atrasado a ataques críticos e limitando sua conversão em vitórias.

O achado central consolida, portanto, a dissociação entre a fidelidade de imitação estatística e o desempenho em combate real: o melhor imitador nas métricas estáticas do *dataset* (MLP com Janela Temporal) mostrou-se o pior jogador na arena, ao passo que a CNN 1-D e a MLP Pura equilibraram de forma mais eficiente as dimensões de imitação e dinâmica de jogo.

4 Conclusão e Trabalhos Futuros

Este trabalho apresentou um estudo comparativo de quatro arquiteturas de redes neurais para o controle autônomo de um agente em *Hollow Knight* via *Behavior Cloning*, utilizando um *dataset* de 200 demonstrações humanas. Nas métricas estáticas de classificação, os modelos apresentaram desempenho equivalente, com o *F1-score micro* estabilizado entre 0,85 e 0,86. Contudo, a avaliação prática revelou a dissociação entre a fidelidade de imitação e o desempenho em combate: a MLP com Janela Temporal, apesar de liderar a validação estática devido ao superajuste pelo vazamento de fronteira temporal, obteve a pior resiliência em jogo. Em contrapartida, a CNN 1-D e a MLP Pura equilibraram de forma eficiente as dimensões de imitação e dinâmica de jogo, alcançando a maior taxa de sucesso na arena (13,33% de vitórias cada), destacando-se a MLP Pura pelo menor custo-benefício computacional.

O diagnóstico das métricas também orientou otimizações de engenharia essenciais no pipeline do sistema. A reestruturação do carregamento de dados eliminou processamentos redundantes e reduziu em 75% o tempo de experimentação. Adicionalmente, a exclusão

de ações nulas combinada a estratégias severas de regularização (*dropout*, *weight decay*, *scheduler* e *gradient clipping*) mitigou o superajuste e refinou o *f1-score macro*. Por fim, a aplicação de *early stopping* e a redução do limiar de decisão de 0,5 para 0,2 compensaram o perfil conservador induzido pelas redes, otimizando a responsividade física do agente em jogo real.

Como trabalhos futuros, propõe-se: (i) tratar o desbalanceamento das classes raras via pesos dinâmicos (`pos_weight`) na perda para corrigir o acionamento do comando "Cima"; (ii) conduzir testes empíricos em ambiente com aceleração por GPU CUDA nativa para eliminar o gargalo de latência sofrido pelo fluxo recorrente da GRU em CPU; (iii) avaliar janelas temporais maiores e a supressão de velocidades explícitas no vetor de estado; (iv) incluir na comparação a arquitetura *Vanilla RNN* já implementada no repositório; e (v) investigar o hiato entre imitação e combate combinando o *Behavior Cloning* com ajuste fino por Aprendizado por Reforço guiado pela vitória.

Referências

1 POMERLEAU, D. A. Alvinn: An autonomous land vehicle in a neural network. In: *Advances in Neural Information Processing Systems (NIPS)*. [S.l.: s.n.], 1989.

2 BAIN, M.; SAMMUT, C. A framework for behavioural cloning. *Machine Intelligence*, v. 15, 1995.

3 CHRISTOFOLETTI, V. H. B. *Hollow-Knight-Neural-Network: A neural network model capable of playing Hollow Knight to defeat the Hornet Protector at Hall of Gods*. [S.l.]: GitHub, 2026. <https://github.com/Vitao2/Hollow-Knight-Neural-Network>. Acessado em: 10 de maio de 2026.

4 PASZKE, A. et al. Pytorch: An imperative style, high-performance deep learning library. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2019.

5 PEDREGOSA, F. et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, v. 12, 1011.